MicroPython on Raspberry Pi Pico W Lab Manual

I. Objective

By the end of this session, students will be able to set up a **MicroPython Web Server** on the Raspberry Pi Pico W to create and consume a basic RESTful API for on-board sensor reading (temperature) and actuator control (LED).

II. Prerequisites and Setup

A. Hardware & Software

- Raspberry Pi Pico W board.
- Micro-USB cable for power and data.
- Computer with a **Thonny** IDE (for MicroPython development).
- The Pico W must be running the **latest MicroPython firmware**.

B. Installing MicroPython

1. If not already done, hold the **BOOTSEL** button while connecting the Pico W to your computer.
2. The Pico W should appear as a drive named **RPI-RP2**.
3. Drag and drop the official MicroPython UF2 file onto this drive. The drive will disappear, and the Pico W will reboot running MicroPython.
4. Open **Thonny** and ensure it is configured to talk to the Pico W. (Bottom right, select "MicroPython (Raspberry Pi Pico)")

III. MicroPython REST API Setup

The RESTful API logic will be broken into two main files:

- `main.py` (the main program and API router) and
- `web_server.py` (the core HTTP server logic).

The **API router** in main.py is responsible for defining and managing the routes of your API. It maps incoming HTTP requests (such as GET /users or POST /orders) to the appropriate handler functions that implement the business logic. In other words, it acts like a traffic controller, ensuring each request is directed to the correct endpoint based on its URL path and HTTP method.

The **core HTTP server logic** in web_server.py handles the low-level networking tasks required to run the server. This includes listening on a specific port, accepting client connections, parsing raw HTTP requests, and sending back responses. Essentially, it provides the foundation for communication over HTTP, while the API router builds on top of it to deliver structured RESTful functionality.

A. Utility Code (`web_server.py`)

This file contains the simple socket server and should be uploaded to the Pico W first.

- Implements HTTP parsing and routing logic.
- Defines helper functions:
  - http_response() – Builds HTTP response strings.
  - _parse_request() – Extracts method, path, headers, and body.

    - _parse_led_state() – Reads LED state from JSON, form data, or plain text.
- Handles routes:
    - GET / – API root page.
    - GET /temp – Returns current temperature.
    - POST /led – Sets LED state (ON/OFF) based on request body.
- Routing logic:
    - If GET /temp → Respond with temperature.
    - If POST /led → Parse body for state (0 or 1) and return LED status.
    - If unknown path → Respond with 404 Not Found.

```python
# web_server.py

# Flexible HTTP response builder
def http_response(body, status=200, content_type="text/html"):
    reason = {
        200: "OK",
        400: "Bad Request",
        404: "Not Found",
        405: "Method Not Allowed",
    }.get(status, "OK")

    return (
        f"HTTP/1.1 {status} {reason}\r\n"
        f"Content-Type: {content_type}\r\n"
        f"Connection: close\r\n"
        f"\r\n"
        f"{body}"
    )

def _parse_request(raw):
    """
    Very simple HTTP request parser for method, path, headers, and body.
    Assumes CRLF separators and small payloads (fits in initial recv).
    """
    # Split header and body
    parts = raw.split("\r\n\r\n", 1)
    header_block = parts[0]
    body = parts[1] if len(parts) > 1 else ""

    # First line: METHOD PATH HTTP/VERSION
    header_lines = header_block.split("\r\n")
    request_line = header_lines[0] if header_lines else ""
    try:
        method, path, _ = request_line.split(" ", 2)
    except ValueError:
        method, path = "GET", "/"

    # Parse headers into dict (case-insensitive keys)
    headers = {}
    for line in header_lines[1:]:
        if ":" in line:
            k, v = line.split(":", 1)
            headers[k.strip().lower()] = v.strip()

    return method.upper(), path, headers, body

def _parse_led_state(body, content_type):
    """
    Accepts JSON: {"state": 1}, form: state=1, or plain text: '1'
    Returns 0 or 1, or None if invalid.
    """
    # Normalize content_type
    ct = (content_type or "").split(";")[0].strip().lower()

    # JSON
    if ct == "application/json":
        try:
            import json
            data = json.loads(body)
            state = data.get("state", None)
```

```
            state = data.get("state", None)
            if state in (0, 1):  # accept explicit 0/1
                return state
            # also allow truthy/falsy
            if isinstance(state, bool):
                return 1 if state else 0
            if isinstance(state, str) and state in ("0", "1"):
                return int(state)
        except Exception:
            return None

    # Form-encoded
    elif ct == "application/x-www-form-urlencoded":
        try:
            # Very simple parse without url-decoding
            pairs = body.split("&")
            for p in pairs:
                if "=" in p:
                    k, v = p.split("=", 1)
                    if k == "state" and v in ("0", "1"):
                        return int(v)
        except Exception:
            return None

    # Plain text (or unknown content type)
    else:
        txt = body.strip()
        if txt in ("0", "1"):
            return int(txt)

    return None

# Function to serve the client connection
def serve_client(client_socket, temp_c):
    request = client_socket.recv(1024).decode("utf-8")

    method, path, headers, body = _parse_request(request)

    # LED state sentinel: -1 means "no change"
    global led_state
    led_state = -1

    # --- Routing ---
    if path == "/temp":
        if method != "GET":
            response = http_response(
                "<h1>Method Not Allowed</h1><p>Use GET for /temp</p>",
                status=405
            )
        else:
            response_html = (
                f"<h1>Pico W Temperature</h1>"
                f"<p>Temperature: {temp_c:.2f} &deg;C</p>"
            )
            response = http_response(response_html, status=200)
            # optional: reading temp does not change LED
            led_state = -1

    elif path == "/led":
        if method != "POST":
            response = http_response(
                "<h1>Method Not Allowed</h1><p>Use POST for /led</p>",
                status=405
            )
        else:
            ct = headers.get("content-type", "")
            state = _parse_led_state(body, ct)
            if state is None:
                response = http_response(
                    (
```

```
                    "<h1>Bad Request</h1>"
                    "<p>Send LED state via JSON {'state': 0|1}, "
                    "form 'state=0|1', or plain text '0'/'1'.</p>"
                ),
                status=400
            )
            led_state = -1
        else:
            led_state = state
            status_text = "ON" if state == 1 else "OFF"
            response_html = (
                f"<h1>LED Control</h1><p>LED is {status_text}</p>"
            )
            response = http_response(response_html, status=200)

    elif path == "/":
        if method != "GET":
            response = http_response(
                "<h1>Method Not Allowed</h1><p>Use GET for /</p>",
                status=405
            )
        else:
            response_html = (
                "<h1>Pico W API Root</h1>"
                "<p>Available Endpoints:</p>"
                "<ul>"
                "<li>GET /temp</li>"
                "<li>POST /led (body: {'state':0|1}, or state=0|1, or '0'/
                "</ul>"
            )
            response = http_response(response_html, status=200)
            led_state = -1

    else:
        response = http_response(
            "<h1>Not Found</h1><p>Unknown path</p>",
            status=404
        )
        led_state = -1

    # Send response and close
    try:
        client_socket.send(response.encode("utf-8"))
    finally:
        client_socket.close()

    return led_state
```

B. Main Application Logic (`main.py`)

- Handles Wi-Fi connection and server loop.
- Creates a socket that listens on port 80.
- Accepts incoming client connections and delegates request handling to web_server.py.
- Reads the Pico W internal temperature sensor before serving each request.
- Applies LED state changes returned by serve_client().

Key responsibilities:

- Connect to Wi-Fi using SSID and password.
- Start the HTTP server socket.
- Loop forever:
  - Accept client connection.
  - Read temperature.
  - Call serve_client() to process the request.
  - Update LED state based on the returned value.

```
# main.py
```

```python
# main.py
import network
import socket
import machine
import time
from web_server import serve_client

# WiFi Credentials (CHANGE THESE!)
SSID = "YOUR_WIFI_SSID"
PASSWORD = "YOUR_WIFI_PASSWORD"

# Pico W Peripherals
onboard_led = machine.Pin("LED", machine.Pin.OUT)
temp_sensor = machine.ADC(4)
conversion_factor = 3.3 / (65535)

def wifi_connect():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    wlan.connect(SSID, PASSWORD)

    # Wait for connect or fail
    max_wait = 10
    while max_wait > 0:
        if wlan.status() < 0 or wlan.status() >= 3:
            break
        max_wait -= 1
        print('waiting for connection...')
        time.sleep(1)

    if wlan.status() != 3:
        raise RuntimeError('Network connection failed')
    else:
        status = wlan.ifconfig()
        print('Connected! IP Address:', status[0])
        return status[0]

def read_temperature():
    # Pico W internal temperature sensor reading logic
    voltage = temp_sensor.read_u16() * conversion_factor
    temperature = 27 - (voltage - 0.706) / 0.001721
    return temperature

# --- Main Program ---
try:
    ip_address = wifi_connect()
except Exception as e:
    print(f"Error connecting to WiFi: {e}")
    # Flash LED to indicate failure
    for i in range(5):
        onboard_led.toggle()
        time.sleep(0.2)
    raise

# Create Socket
addr = socket.getaddrinfo('0.0.0.0', 80)[0][-1]
s = socket.socket()
s.bind(addr)
s.listen(1)

print('Listening on:', addr)

# Server Loop
while True:
    try:
        # Check for client connection
        client_conn, client_addr = s.accept()
        print('Client connected from', client_addr)

        # Get current temperature before serving
        current_temp_c = read_temperature()
```

```
        current_temp_c = read_temperature()

        # Serve the request and get the desired LED state
        new_led_state = serve_client(client_conn, current_temp_c)

        # Apply actuator change if requested
        if new_led_state == 0:
            onboard_led.value(0) # LED OFF
        elif new_led_state == 1:
            onboard_led.value(1) # LED ON

    except OSError as e:
        # Catch connection errors and continue loop
        if e.args[0] == 110: # ETIMEDOUT (non-blocking socket)
            pass
        else:
            print('Connection error:', e)
            continue
```

How It Works

The project consists of two main files:

`main.py`

- Connects the Pico W to Wi-Fi using the provided SSID and password.
- Creates a socket that listens on port `80` for incoming HTTP requests.
- Reads the onboard temperature sensor before serving each request.
- Delegates request handling to `serve_client()` in `web_server.py`.
- Applies LED state changes returned by `serve_client()`:
  - `0` → LED OFF
  - `1` → LED ON
  - `-1` → No change

`web_server.py`

- Parses incoming HTTP requests (method, path, headers, body).
- Routes requests to the correct handler:
  - `GET /` → Returns API root page with available endpoints.
  - `GET /temp` → Returns current temperature in HTML.
  - `POST /led` → Sets LED state based on request body.
- Builds HTTP responses using `http_response()`.

Routing Logic:

- If `GET /temp` → Respond with temperature.
- If `POST /led` → Parse body for `state` (0 or 1) and return LED status.
- If unknown path → Respond with `404 Not Found`.

IV. Testing the RESTful API

1. Upload Code

- Upload both `web_server.py` and `main.py` to the Raspberry Pi Pico W using Thonny IDE (ensure these two scripts are saved in the Pico W).
- Ensure you have updated the `SSID` and `PASSWORD` variables in `main.py` with your local WiFi credentials.

2. Run and Connect

- Run `main.py` on the Pico W.
- Wait for the "Connected! IP Address:" message in the Thonny shell. Note down the

displayed IP address (e.g., `192.168.1.100`).

3. Test Endpoints

Use a tool like Postman to interact with your Pico W using its IP address.

Method	Endpoint	Description	Example Response
GET	`/`	API root page with available routes	HTML listing `/temp` and `/led`
GET	`/temp`	Returns current temperature	`<h1>Pico W Temperature</h1> <p>Temperature: 28.34 °C</p>`
POST	`/led`	Sets LED state (ON/OFF)	`<h1>LED Control</h1><p>LED is ON</p>` or OFF

POST Body Options for `/led`:

- **JSON:**

  ```
  {"state": 1}
  ```

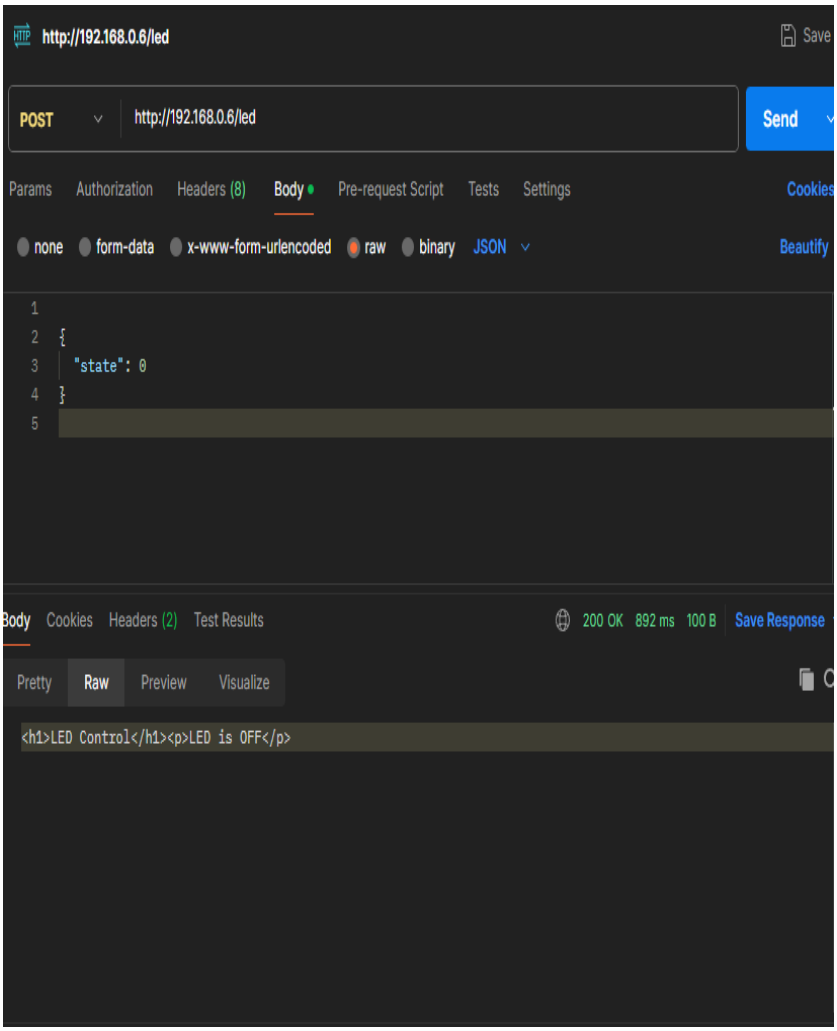
  (Use `1` for ON, `0` for OFF)
- **Form:** state=1
- **Plain text:** 1

Using Postman

GET Requests

1. Open Postman and create a new request.
2. **Method:** `GET`
3. **URL:**
   - For root page: `http://<Pico-IP>/`
   - For temperature: `http://<Pico-IP>/temp`
4. Click **Send**.
   - `/` returns the API root HTML.
   - `/temp` returns the current temperature.

POST Request to Toggle LED



1. Open Postman and create a new request.
2. **Method:** `POST`
3. **URL:** `http://<Pico-IP>/led`
4. Go to **Body** tab:
   - Select **raw**.
   - Choose **JSON** from the dropdown.
   - Enter:

     ```
     {
       "state": 1
     }
     ```

     (Use `1` for ON, `0` for OFF)
5. Ensure header `Content-Type: application/json` is set (Postman adds this automatically when JSON is selected).
6. Click **Send**.
   - Response (raw) will confirm LED status:`<h1>LED Control</h1><p>LED is ON</p>`

Alternative:
Use **x-www-form-urlencoded** in Body tab:

```
Key: state
Value: 1
```

Assignment for this Lab Session